

Relatório Técnico - CSRF (Cross-Site Request Forgery)

Disciplina: Segurança de Sistemas

Integrantes: João Pedro Ribeiro / Hugo Gabriel Sales Soares

Introdução

A segurança em aplicações web é um campo em constante evolução, no qual novas vulnerabilidades são descobertas e exploradas diariamente. Dentre os ataques mais persistentes e documentados, o Cross-Site Request Forgery (CSRF) ocupa posição de destaque no OWASP Top 10, classificado dentro da categoria A01 - Broken Access Control.

O CSRF é um tipo de ataque que explora a confiança que um servidor deposita no navegador de um usuário autenticado. Diferentemente de outros ataques que comprometem diretamente o sistema, o CSRF usa o próprio navegador da vítima como veículo, forçando-o a executar ações não autorizadas em um site no qual ela já está autenticada.

Este relatório documenta o estudo da vulnerabilidade CSRF, sua demonstração prática em ambiente controlado e as técnicas de prevenção aplicadas. Para isso, foi desenvolvida uma aplicação bancária propositalmente vulnerável chamada **SafeBank**, acompanhada de um site malicioso que simula o ataque, e uma versão corrigida do mesmo banco com todas as proteções implementadas.

Descrição da Vulnerabilidade

O que é CSRF

Cross-Site Request Forgery é um ataque no qual um site malicioso induz o navegador de um usuário autenticado a enviar requisições não intencionais a outro site no qual a vítima possui uma sessão ativa. O servidor alvo não consegue distinguir a requisição legítima da forjada, pois o navegador inclui automaticamente os cookies de sessão em qualquer requisição ao domínio correspondente.

Mecanismo técnico

O ataque se baseia em três comportamentos padrão dos navegadores e servidores web:

- **Envio automático de cookies:** o navegador anexa todos os cookies válidos de um domínio em qualquer requisição a esse domínio, independentemente da origem que iniciou a requisição.
- **Ausência de verificação de origem no servidor:** servidores que autenticam requisições apenas pelo cookie de sessão não têm como saber se aquela requisição foi iniciada pelo próprio usuário ou por um terceiro.
- **Formulários HTML sem restrição de origem:** qualquer página pode submeter um formulário HTML para qualquer URL, e o navegador executa a submissão sem questionar a origem.

Condições necessárias

Para que o ataque seja bem-sucedido, três condições precisam ser verdadeiras simultaneamente:

- A vítima deve estar autenticada no site alvo no momento do ataque;
- O cookie de sessão não deve ter o atributo `SameSite=Strict`;
- O endpoint alvo não deve validar a origem da requisição nem exigir um token CSRF.

Cenário Vulnerável

A aplicação SafeBank (vulnerável)

O SafeBank é uma aplicação bancária desenvolvida em Next.js que simula operações financeiras básicas: login, visualização de saldo, transferência entre contas e alteração de e-mail. A versão vulnerável foi construída intencionalmente sem proteções CSRF para fins didáticos.

Contas disponíveis no laboratório:

Usuário	Senha	Saldo inicial
alice	alice123	R\$ 10.000,00
bob	bob123	R\$ 500,00

Vulnerabilidade 1 - Cookie sem SameSite

No processo de login, o servidor cria um cookie de sessão sem o atributo `SameSite`:

```
// safebank-vulneravel/app/api/auth/login/route.ts - linhas 17-20
response.cookies.set('session_id', sessionId, {
  httpOnly: true,
  path: '/',
  // SameSite ausente: o navegador envia este cookie em qualquer
  // requisição para localhost:3000, mesmo originada de outro
  site.
})
```

A ausência do atributo `SameSite` faz com que o navegador utilize o comportamento padrão, que em navegadores modernos é equivalente a `SameSite=Lax`. No entanto, como ambos os domínios do laboratório estão em `localhost`, o navegador os trata como o mesmo site registrável, tornando a proteção do `Lax` ineficaz neste cenário - o que reforça a necessidade de `SameSite=Strict`.

Vulnerabilidade 2 - Endpoint sem proteção CSRF

O endpoint de transferência autentica o usuário apenas pelo cookie de sessão, sem verificar a origem da requisição nem exigir um token secreto:

```
// safebank-vulneravel/app/api/transfer/route.ts - linhas 4-13
export async function POST(request: NextRequest) {
  const sessionId = request.cookies.get('session_id')?.value
```

```

    if (!sessionId) {
        return NextResponse.json({ error: 'Não autenticado' }, {
status: 401 })
    }

    const userId = sessions.get(sessionId)
    // Não verifica de onde veio a requisição.
    // Não exige token CSRF.
    // Qualquer POST com cookie válido é processado.

```

O endpoint também aceita o tipo de conteúdo `application/x-www-form-urlencoded`, que é o formato padrão de formulários HTML, permitindo que o ataque seja executado sem nenhum JavaScript do lado da vítima.

O site do atacante

O site malicioso simula uma página de premiação falsa chamada "PrêmiosBrasil", com roleta animada, contador regressivo e depoimentos fabricados. A engenharia social tem como objetivo manter a atenção da vítima enquanto o ataque ocorre em segundo plano.

Por trás da interface, um formulário HTML oculto e um iframe invisível executam o ataque no momento em que a roleta começa a girar:

```

<!-- csrf-attacker/index.html - linhas 464-474 -->
<form id="attack-vulnerable"
action="http://localhost:3000/api/transfer"
      method="POST" enctype="application/x-www-form-urlencoded"
      target="vulnerable-frame" style="display:none">
  <input name="to"      value="bob" />
  <input name="amount" value="1000" />
</form>
<iframe name="vulnerable-frame" style="display:none"></iframe>

// csrf-attacker/index.html - linhas 572-580
function spin() {
  // ...
  // Dispara o ataque silenciosamente nos dois alvos
  fireCSRF('vulnerable')
  fireCSRF('secure')
}

```

Demonstração em Ambiente Controlado

Toda a demonstração ocorre exclusivamente em ambiente local, sem qualquer interação com sistemas externos. Os dois projetos rodam em portas diferentes do `localhost`.

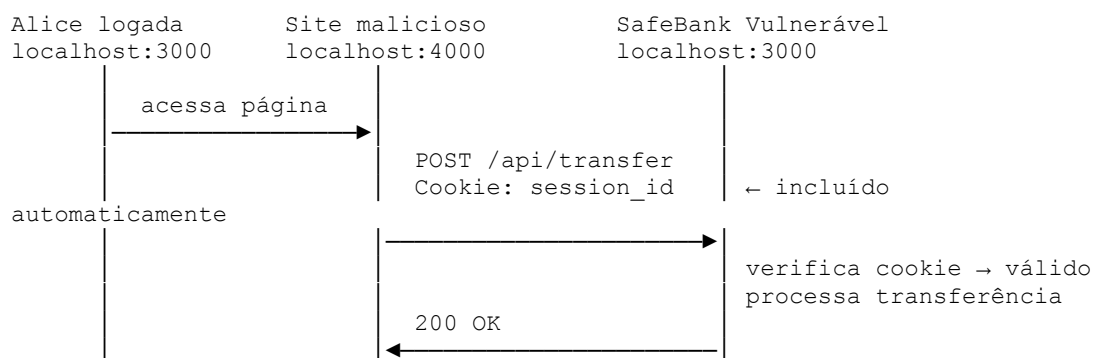
Infraestrutura do laboratório

Componente	Endereço	Descrição
SafeBank Vulnerável	http://localhost:3000	Banco sem proteção CSRF
SafeBank Seguro	http://localhost:3001	Banco com todas as proteções
Site do Atacante	http://localhost:4000	Página maliciosa "PrêmiosBrasil"

Passo a passo do ataque

1. Iniciar o SafeBank vulnerável na porta 3000;
2. Acessar `localhost:3000` e fazer login como **alice**;
3. Verificar que o saldo inicial é de **R\$ 10.000,00**;
4. Sem fazer logout, abrir `localhost:4000` em outra aba do mesmo navegador;
5. O pop-up da roleta aparece automaticamente e inicia a animação;
6. No momento em que a roleta gira, o formulário oculto é submetido silenciosamente;
7. O navegador inclui o cookie `session_id` de alice automaticamente na requisição;
8. O SafeBank recebe um POST válido para `/api/transfer` e processa a transferência;
9. Voltar para `localhost:3000` e recarregar a página;
10. Constatar que o saldo de alice diminuiu **R\$ 1.000,00** sem qualquer ação intencional dela.

Fluxo da requisição forjada



Impactos Possíveis

Impactos diretos

Em um sistema real, o CSRF pode ser utilizado para executar qualquer ação que o usuário autenticado tenha permissão de realizar, incluindo:

- **Transferências financeiras não autorizadas**, causando prejuízo direto à vítima;
- **Alteração de e-mail ou senha**, resultando em perda de acesso permanente à conta;
- **Compras, pedidos ou cancelamentos** realizados sem consentimento;
- **Execução de ações administrativas** por usuários comuns em sistemas com painel de administração vulnerável;
- **Exclusão de dados** como arquivos, registros ou configurações importantes.

Gravidade

A gravidade do ataque depende diretamente das ações disponíveis para o usuário autenticado. Em sistemas bancários, e-commerces ou plataformas com dados sensíveis, o impacto pode ser financeiro, reputacional e legal. O ataque é especialmente perigoso porque:

- Não deixa rastros evidentes no sistema da vítima;
- Não requer que a vítima instale nada ou forneça credenciais;
- Pode ser executado por qualquer site que a vítima visite enquanto está autenticada.

Casos reais documentados

Sistema	Ano	Impacto
Netflix	2006	Permitia alterar e-mail e senha da conta
ING Direct	2008	Transferências bancárias não autorizadas
YouTube	2008	Inscrições em canais e adição de favoritos em massa
Gmail	2007	Criação silenciosa de filtros para desvio de e-mails

Correção Aplicada

A versão corrigida do SafeBank (`safebank-seguro`, porta 3001) implementa três camadas de defesa independentes. Cada camada é suficiente para bloquear o ataque isoladamente; juntas, formam uma proteção robusta.

Camada 1 - Cookie SameSite=Strict

O atributo `SameSite=Strict` instrui o navegador a nunca enviar o cookie em requisições que partem de outro site:

```
// safebank-seguro/app/api/auth/login/route.ts - linhas 27-32
response.cookies.set('session_id', sessionId, {
  httpOnly: true,
  path: '/',
  sameSite: 'strict', // cookie não é enviado em cross-site
  secure: process.env.NODE_ENV === 'production',
})
```

Com esta configuração, quando o formulário do site atacante tenta enviar uma requisição para o SafeBank, o navegador simplesmente não inclui o cookie - o servidor recebe a requisição sem autenticação e retorna 401.

Camada 2 - Validação de Origin/Referer

O servidor verifica o header `Origin` de toda requisição sensível. O navegador sempre inclui este header em POSTs cross-origin, e seu valor não pode ser falsificado por JavaScript de outro domínio:

```
// safebank-seguro/lib/csrf.ts - linhas 35-58
export function validateRequestOrigin(request: NextRequest) {
  const expectedOrigin = new URL(request.url).origin
  const origin = request.headers.get('origin')
  const referer = request.headers.get('referer')

  if (origin) {
    return origin === expectedOrigin
  }
  if (referer) {
    try {
      return new URL(referer).origin === expectedOrigin
    } catch {
      return false
    }
  }
  return false // falha fechada: sem Origin/Referer, rejeita
}
```

Se a requisição vier de localhost:4000, o header Origin: http://localhost:4000 não bate com http://localhost:3001 e a requisição é rejeitada com status 403.

Camada 3 - Token CSRF por Sessão

No momento do login, um token criptograficamente aleatório de 32 bytes é gerado e associado à sessão no servidor:

```
// safebank-seguro/lib/csrf.ts - linhas 21-25
export function createCsrfTokenForSession(sessionId: string) {
  const token = randomBytes(32).toString('base64url')
  csrfTokens.set(sessionId, token)
  return token
}
```

O token é entregue ao frontend apenas quando o dashboard autenticado é renderizado, como prop de um Server Component. O frontend legítimo envia o token em um header customizado X-CSRF-Token em toda requisição sensível:

```
// safebank-seguro/app/dashboard/transfer-form.tsx - linhas 18-26
const res = await fetch('/api/transfer', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'X-CSRF-Token': csrfToken, // token que só a página legítima
    conhece
  }
})
```

```

},
body: JSON.stringify({ to, amount: Number(amount) }),
})

```

O endpoint valida o token usando `timingSafeEqual`, que previne ataques de timing na comparação:

```

// safebank-seguro/lib/csrf.ts - linhas 92-105
const expectedToken = csrfTokens.get(sessionId)
const receivedToken = request.headers.get('x-csrf-token')

if (!expectedToken || !receivedToken ||
!safeTokenEquals(receivedToken, expectedToken)) {
  return {
    ok: false,
    response: NextResponse.json({ error: 'Token CSRF invalido' },
{ status: 403 })),
  }
}

```

Um formulário HTML puro não consegue definir headers customizados - apenas `fetch` ou `XMLHttpRequest` têm essa capacidade. Como o atacante não conhece o valor do token, não consegue montar o header correto e a requisição é bloqueada com 403.

Boas Práticas Preventivas

Com base na implementação realizada e nas diretrizes do OWASP, as seguintes boas práticas devem ser adotadas para prevenir CSRF em aplicações web:

- **Usar `SameSite=Strict` nos cookies de sessão:** impede que o navegador envie o cookie em qualquer contexto cross-site. Em casos onde `Strict` seja incompatível com o fluxo de navegação, `SameSite=Lax` oferece proteção parcial.
- **Implementar token CSRF único por sessão:** gerar um token criptograficamente aleatório (mínimo 32 bytes) associado à sessão do usuário e exigi-lo em toda ação que modifica estado no servidor.
- **Validar o header `Origin` ou `Referer`:** verificar no backend se a requisição parte da própria origem do servidor antes de processá-la.
- **Usar os métodos HTTP corretos:** operações que alteram estado (transferências, exclusões, atualizações) devem usar `POST`, `PUT` ou `DELETE` - nunca `GET`. Requisições `GET` não devem ter efeitos colaterais.
- **Configurar `HttpOnly` e `Secure` nos cookies:** `HttpOnly` impede que JavaScript acesse o cookie (mitigando XSS combinado com CSRF); `Secure` garante que o cookie só seja enviado via HTTPS.
- **Exigir re-autenticação para ações críticas:** operações de alto risco como alteração de senha, exclusão de conta ou transferências acima de determinado valor devem exigir confirmação de identidade adicional.
- **Manter HTTPS em produção:** sem HTTPS, tokens e cookies podem ser interceptados em trânsito, tornando todas as outras proteções ineficazes.

- **Nunca confiar apenas no frontend:** verificações de autorização e validação de origem devem sempre estar no backend. Esconder botões ou desabilitar campos no frontend não constitui proteção.

Conclusão

Este trabalho demonstrou que o CSRF é uma vulnerabilidade real, de fácil exploração e com impactos potencialmente severos em aplicações que não adotam as proteções adequadas. A aplicação SafeBank vulnerável comprovou que um ataque silencioso pode ser executado com apenas um formulário HTML oculto, sem nenhuma interação consciente da vítima.

A versão corrigida do SafeBank demonstrou que a proteção efetiva contra CSRF não depende de uma única medida, mas da combinação de camadas complementares: o atributo `SameSite=Strict` no cookie de sessão, a validação do header `Origin` no servidor e a implementação de tokens CSRF criptograficamente seguros.

O principal aprendizado deste estudo é que a segurança deve ser projetada desde a concepção da aplicação, e não adicionada como correção posterior. Mecanismos simples como a configuração correta de cookies e a adoção de tokens CSRF são suficientes para eliminar completamente esta classe de vulnerabilidade, desde que aplicados de forma consistente em todas as rotas sensíveis do sistema.

Referências

OWASP FOUNDATION. **Cross-Site Request Forgery (CSRF)**. OWASP, 2021. <https://owasp.org/www-community/attacks/csrf>

OWASP FOUNDATION. **Cross-Site Request Forgery Prevention Cheat Sheet**. OWASP Cheat Sheet Series, 2021. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

OWASP FOUNDATION. **OWASP Top 10 - A01:2021 Broken Access Control**. OWASP, 2021. https://owasp.org/Top10/A01_2021-Broken_Access_Control

MDN WEB DOCS. **Using HTTP cookies**. Mozilla Developer Network, 2024. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies>

MDN WEB DOCS. **Same-origin policy**. Mozilla Developer Network, 2024. https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy

MDN WEB DOCS. **Fetch API - Using Fetch**. Mozilla Developer Network, 2024. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

WEST, M. et al. **Cookies: HTTP State Management Mechanism (RFC 6265bis)**. IETF, 2019. <https://datatracker.ietf.org/doc/html/draft-ietf-httpbis-rfc6265bis>